

# Assignment 2: iPerf, TCP Statistics, and Congestion Control

CS 422: Computer Networks, Fall 2026

Instructor: Vamsi Addanki

TAs: Youngsuk Kim, Xiao Luo, and Albert Vo

**Due Date: October 21, 2026 @ 11:45 PM Eastern Time**

## 1. iPerf Throughput Application (20 points)

- (a) **Write a socket program from scratch (Python).** Implement a Python client that opens a TCP connection to the destination and continuously sends application data for a fixed experiment duration (e.g., 60 seconds per destination).
- **iPerf3 server compatibility.**  
You must implement a Python TCP client that can correctly communicate with publicly available `iperf3` servers listed at: <https://iperf3serverlist.net/>.  
**Important:** You are NOT allowed to run your own custom receiver to simplify the protocol. Your client must successfully complete a TCP test with a standard `iperf3` server.
  - Your implementation must:
    - (i) Establish the control connection.
    - (ii) Perform the JSON-based parameter exchange required by the `iperf3` protocol.
    - (iii) Open the data connection.
    - (iv) Transmit data continuously for a configurable duration.
    - (v) Properly terminate the test following `iperf3` semantics.
  - Reverse-engineering guidance:
    - You may inspect the official `iperf3` source code to understand the protocol structure.
    - You may capture packet traces using `tcpdump` or `wireshark` to analyze the handshake and message exchange.
    - However, you may not invoke the `iperf3` binary itself to run the experiment.
    - You can also take help from an LLM. Be sure to verify any generated code and understand it fully, as LLMs can produce incorrect or non-robust implementations.
  - Your client must robustly handle:
    - Non-responsive or overloaded servers,
    - Servers that reject parameter negotiation,
    - Connection timeouts,
    - Premature connection termination.
- (b) **Destination selection.** From <https://iperf3serverlist.net/>, pick  $n$  random destination servers, with  $n$  configurable as a command-line argument. A reasonable default is  $n = 10$ , but you may choose more or fewer. Record the selected server addresses and ports.
- If a server rejects or rate-limits the test, log the failure and try a replacement, with a bounded number of attempts. Report any shortfall in the requested destination count.
- (c) **Estimate acknowledged throughput (goodput).** At regular intervals (e.g., every 200 ms or 1 s), compute and record:

$$\text{goodput}(t) = \frac{\Delta \text{bytes acknowledged}}{\Delta t} \cdot 8 \quad [\text{bits/s}]$$

Use differences between successive cumulative acknowledged-byte counters and the actual elapsed sampling time. Obtain TCP statistics from the data socket using `getsockopt()`. Plot throughput over time for all destinations (separate panels are fine), and provide a summary table of minimum, median, mean, and 95th-percentile throughput for each destination and algorithm.

## 2. TCP Stats Tracing During Transfer (40 points)

For this question, use CUBIC or your operating system's default TCP congestion control algorithm. Record the algorithm actually used and use it for all measurements, plots, and observations in this question.

- (a) **Extract TCP socket statistics periodically.** While your client is sending, at the same sampling interval as goodput, extract TCP-layer statistics from the live data socket, rather than the `iperf3` control socket.
- **Required:** a timestamp, `snd_cwnd` (hint: `TCP_INFO / TCP_INFO_FMT`), RTT estimate (e.g., `srtt`), and a loss signal (e.g., retransmits, lost, or delivered/retrans counters depending on what your platform exposes). For each sample in the log, also record the goodput (similar to Q1) alongside other TCP stats.
  - **Recommended:** Also log RTT variation (`rttvar`), pacing rate, cumulative byte counters, and any available delivery-rate estimate.

Store measurements in CSV or JSON, including units, destination, algorithm, and run identifier. Document the decoding format used for your host kernel and how you compute interval measurements and the loss estimate or proxy.

- (b) **Visualization.** Generate PDF plots automatically for a representative destination, showing:
- (i) Time series: `snd_cwnd`, RTT, loss proxy, and throughput (separate plots are fine).
  - (ii) Scatter plots showing relationships:
    - `snd_cwnd` vs goodput,
    - RTT vs goodput,
    - loss signal (e.g., # retransmissions or # timeouts) vs goodput.

*Note: Without further configurations, these runs will use the default TCP congestion control algorithm (e.g., CUBIC in Linux Kernel).*

- (c) **Observations.** Explain what each of the above TCP metrics (in the visualizations) means and how you expect it to influence goodput in congestion avoidance, and describe any anomalous behavior observed in your traces.

## 3. Compare Algorithms and Design a Congestion Control Algorithm (40 points)

- (a) **Compare CUBIC, Reno, and BBR and identify operating regimes.** Repeat the measurements from Questions 1 and 2 with each of CUBIC, Reno, and BBR. Select the algorithm on the data socket before connecting, and record the algorithm actually used. Repeat all visualizations from Question 2(b) for each algorithm: time series of `snd_cwnd`, RTT, loss proxy, and throughput, and scatter plots of congestion window, RTT, and loss signal against goodput. Use the same representative destination for all three algorithms, with clearly labeled curves or separate panels and consistent axis scales to facilitate comparison. Include these plots in your report. Use the same destinations, transfer duration, and sampling interval for all three algorithms, and repeat the tests across multiple runs. Report the number of repetitions and test order. Identify the network conditions under which each algorithm performs better in throughput and RTT, and explain why using your measurements. Relate your observations to congestion window size, `rtt`, queueing, loss, and the bandwidth-delay product. Discuss variability across runs and the limitations of comparisons over changing public Internet paths.

- (b) **Design a hand-written algorithm.** Using your measurements, design a congestion control algorithm and write pseudocode specifying how it updates the congestion window based on RTT, packet-loss estimates or a loss proxy, and throughput. Explain your design choices. Discuss whether competing flows using your algorithm are expected to share bandwidth fairly and whether their sending rates converge to a stable operating point. State your assumptions and explain any expected limitations.

## Report

Automate the experiments and plotting with a single entry-point script. Accept a server-list file containing addresses and ports, and document command-line options for the number of destinations, transfer duration, sampling interval, and repetitions. Run all three congestion control algorithms, save raw measurements, and generate PDF plots and summary tables. Include all inputs, dependencies, and commands needed to reproduce the results.

Use Docker (“FROM ubuntu:24.04”) to containerize your code and experiments, and include the Dockerfile in your repo. The Docker image should execute the full experiment pipeline on a supported host with the required congestion control algorithms available in its Linux kernel. Record the host kernel version and any required configuration. The Docker container standardizes the user-space environment (Python, libraries, scripts). However, because the container uses the host Linux kernel, TCP behavior and TCP\_INFO fields depend on the host system. This assignment must be executed on:

- a Linux machine (Ubuntu 22.04+ recommended), or
- Windows using WSL2 (which provides a real Linux kernel).

Brightspace submission should be a single PDF. Include a GitHub or GitLab repository link, with links to relevant functions or line ranges for implementation questions. Write observations and interpretations directly in the report, embed the plots, and identify the code that generates them.

AI tools are allowed. Briefly acknowledge how you used them, and be prepared to explain your code, measurements, and design choices during evaluation.

**Note:** Grading is still based on the oral exam during PSO/office hours in the week of *Oct 26*.

**Grace period:** 3 days (72 Hours) grace period is allowed for submission. After the grace period, 25% off for every 24 hours late, rounded up.